

Semana 1: ¿Qué es un computador? Del ábaco al silicio

Guía completa de la sesión

Visión general

Duración total	3 horas (2 h cátedra + 1 h laboratorio analógico)
Objetivo central	Que los estudiantes comprendan qué significa "computar" y que toda computación, por compleja que sea, se reduce a instrucciones simples ejecutadas con precisión
Idea ancla	Un computador no "piensa" — sigue instrucciones. La inteligencia está en quien escribe las instrucciones
Materiales	Proyector, pizarra, tarjetas de cartulina (5 colores), marcadores, sobres, dados (opcional), copias del control diagnóstico

PARTE 1: CÁTEDRA (120 minutos)

Bloque A — Apertura provocadora (15 min)

Pregunta al curso (proyectar en pantalla):

"¿Es una calculadora un computador? ¿Y un termostato? ¿Y un cerebro?"

Dejar que respondan libremente (mano alzada o en voz alta). No corregir aún. Anotar las respuestas en la pizarra en tres columnas: **SÍ / NO / DEPENDE**.

Objetivo: Exponer que no tienen una definición precisa de "computador" — y que eso está bien. Al final de la clase, la tendrán.

Transición: *"Para entender qué es un computador, necesitamos mirar hacia atrás — mucho más atrás de lo que creen."*

Bloque B — Historia de la computación: del ábaco al silicio (35 min)

Recorrer la línea temporal en **cinco estaciones**, dedicando ~7 minutos a cada una. Para cada estación: mostrar una imagen, contar la historia humana detrás del artefacto, y extraer el principio computacional que introduce.

Estación 1: El ábaco (~3000 a.C.)

- Herramienta de cálculo más antigua que sobrevive en uso.
- **Principio:** Representación posicional de cantidades. Los datos necesitan un *soporte físico*.

- **Pregunta al curso:** ¿Por qué no basta con contar con los dedos?

Estación 2: Las máquinas de Babbage (1830s–1870s)

- Charles Babbage: la Máquina Diferencial (cálculo de tablas) y la Máquina Analítica (programable).
- Ada Lovelace: primera programadora — escribió algoritmos para una máquina que nunca se construyó.
- **Principio:** Separación entre *datos* e *instrucciones*. La idea de un programa.
- **Anécdota:** Babbage no logró construir su máquina por limitaciones de ingeniería mecánica de la época. La visión precedió a la tecnología por un siglo.

Estación 3: Alan Turing y la formalización (1936)

- Mención breve (se profundizará en Semana 5): Turing demostró que una máquina muy simple — con una cinta y unas pocas reglas — puede computar *todo* lo que es computable.
- **Principio:** La computación no depende del material — depende de la *lógica*.
- **Pregunta al curso:** ¿Se puede computar con papel y lápiz? (Sí, y lo haremos hoy en el laboratorio.)

Estación 4: ENIAC y los primeros computadores electrónicos (1945–1960)

- ENIAC: 30 toneladas, 18.000 tubos de vacío, programado reconectando cables físicamente.
- Las "computadoras" eran personas (mayoritariamente mujeres) que hacían cálculos a mano.
- Transición: tubos de vacío → transistores → circuitos integrados.
- **Principio:** Velocidad. La electrónica hace lo mismo que el ábaco, pero miles de millones de veces más rápido.

Estación 5: Del microprocesador a tu bolsillo (1970s–hoy)

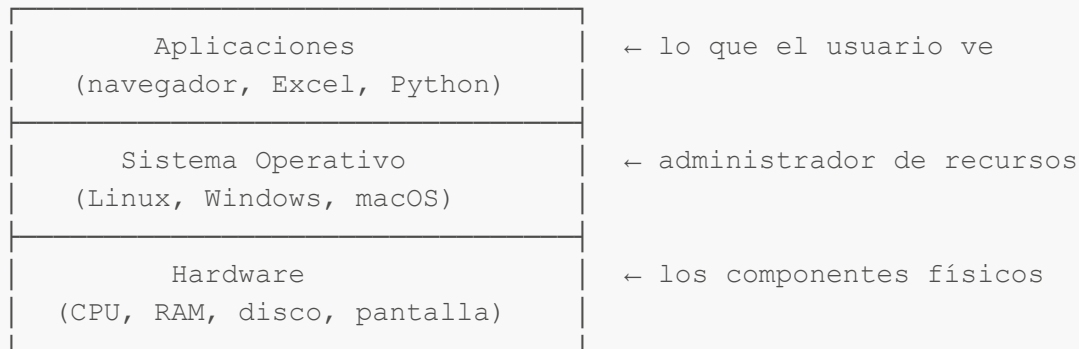
- Intel 4004 (1971): primer microprocesador comercial.
- Ley de Moore: la densidad de transistores se duplica aproximadamente cada dos años (observación empírica, no ley física; ya está llegando a sus límites).
- Tu teléfono tiene más capacidad de cómputo que todas las máquinas que llevaron al ser humano a la Luna.
- **Principio:** Miniaturización y accesibilidad. El computador pasó de llenar una sala a caber en un bolsillo.

Transición: *"Ahora sabemos de dónde vienen. Pero ¿cómo funcionan por dentro?"*

Bloque C — Anatomía de un computador: capas de abstracción (30 min)

El modelo de capas (15 min)

Dibujar en la pizarra un diagrama de capas (de abajo hacia arriba):



- **Hardware:** los componentes físicos que ejecutan operaciones.
- **Sistema operativo:** el intermediario que administra los recursos del hardware y permite que múltiples programas funcionen simultáneamente.
- **Aplicaciones:** los programas que resuelven problemas específicos del usuario.

Analogía ecológica: Es como los niveles de organización biológica. Así como no se puede entender un ecosistema mirando solo las moléculas, tampoco se entiende un programa mirando solo los transistores. Pero las moléculas *sostienen* al ecosistema, igual que el hardware sostiene al software.

Los componentes del hardware (15 min)

Presentar los cuatro componentes fundamentales usando la analogía de una **cocina de restaurante**:

Componente	Función	Analogía
CPU (Unidad Central de Procesamiento)	Ejecuta las instrucciones, hace los cálculos	El chef: sigue la receta paso a paso
Memoria RAM	Almacenamiento rápido y temporal (datos en uso)	El mesón de cocina: lo que el chef tiene a mano ahora
Almacenamiento (disco/SSD)	Almacenamiento permanente	La despensa: todo lo que hay disponible, pero hay que ir a buscarlo
Entrada/Salida (teclado, pantalla, sensores)	Comunicación con el mundo exterior	La ventanilla: por donde entran los pedidos y salen los platos

Dentro de la CPU, hay dos partes clave:

- **ALU** (Unidad Aritmético-Lógica): hace las operaciones matemáticas y comparaciones.
- **Unidad de Control:** lee las instrucciones y coordina quién hace qué y cuándo.

Pregunta al curso: "¿Qué pasa si el chef es rapidísimo pero la mesón enano/diminuto?" → Cuello de botella. La velocidad del sistema depende del componente más lento. Esto es fundamental para entender por qué un computador con mucha CPU pero poca RAM puede ser lento.

Bloque D — ¿Qué significa "computar"? (25 min)

Definición operativa (10 min)

Computar = transformar una entrada (*input*) en una salida (*output*) mediante un procedimiento definido (*algoritmo*), usando un número finito de pasos.

Tres ejemplos concretos:

1. **Sumar dos números:** entrada = (3, 5); procedimiento = suma; salida = 8.
2. **Ordenar una lista de especies por abundancia:** entrada = lista desordenada; procedimiento = algoritmo de ordenamiento; salida = lista ordenada.
3. **Determinar si un área califica como área protegida:** entrada = datos de biodiversidad, superficie, amenazas; procedimiento = criterios UICN; salida = sí/no + categoría.

Punto clave: En los tres casos, el procedimiento debe ser *tan preciso* que cualquiera que lo siga — persona, máquina, o extraterrestre — obtenga el mismo resultado. *Es lo que distingue un algoritmo de una intuición.*

Las tres preguntas fundamentales de la computación (15 min)

Presentar como marco conceptual que acompañará todo el curso:

1. **¿Es computable?** — ¿Existe un procedimiento finito que resuelva este problema? (Semana 5: Turing demostró que hay problemas que *no* son computables.)
2. **¿Qué tan difícil es?** — ¿Cuántos pasos requiere? ¿Escala bien cuando los datos crecen? (Semana 4: complejidad algorítmica.)
3. **¿Cómo lo expreso?** — ¿En qué lenguaje escribo las instrucciones? (Sección 2: Python.)

Analogía ecológica: Es como preguntarse ante un problema de conservación: (1) ¿Es posible restaurar este ecosistema? (2) ¿Cuánto costará en tiempo y recursos? (3) ¿Qué herramientas y métodos concretos usaré?

Bloque E — Cierre y puente al laboratorio (15 min)

Recapitulación rápida (5 min)

Volver a la pregunta de apertura: "*¿Es una calculadora un computador?*"

- Una calculadora simple **no** es un computador en sentido estricto: ejecuta operaciones fijas, no es programable.
- Un termostato moderno (programable) **sí** cumple la definición mínima.
- Un cerebro... es una pregunta abierta que la humanidad aún debate. Pero si la tesis de Church-Turing es correcta, *si* el cerebro computa, una Máquina de Turing puede simularlo.

Anticipación del laboratorio (10 min)

"Ahora van a ser ustedes el computador. Literalmente."

Explicar brevemente la actividad:

- Van a formar grupos de 5 personas.
- Cada persona será un **componente** del computador: Programador, Unidad de Control, ALU, Memoria y E/S.
- El programador escribe instrucciones en tarjetas. Los demás deben ejecutarlas *al pie de la letra* — sin interpretar, sin asumir, sin improvisar.
- Si la instrucción es ambigua o incompleta, el computador **falla**.

"Van a descubrir en carne propia por qué la precisión importa."

PARTE 2: LABORATORIO ANALÓGICO (60 minutos)

"El Computador Humano"

Preparación previa (docente)

Materiales por grupo (5 personas)

- 1 sobre con **tarjetas de instrucciones** en blanco (cartulina blanca, ~10 tarjetas tamaño media carta)
- 1 set de **tarjetas de rol** (5 tarjetas de colores diferentes, ver abajo)
- 1 **hoja de memoria** (tabla impresa con 10 celdas numeradas, simulando registros de memoria)
- 1 marcador
- 1 hoja en blanco para la salida (pantalla)
- 1 dado de 6 caras (para la ronda avanzada, opcional)

Tarjetas de rol

Imprimir en cartulinas de colores distintos. Cada tarjeta lleva el nombre del rol en grande y las reglas en el reverso.

PROGRAMADOR/A

Eres quien diseña la solución. Escribes instrucciones en las tarjetas en blanco, **una instrucción por tarjeta**, y las pasas a la Unidad de Control en orden. NO puedes hablar con los demás componentes una vez que el programa comienza a ejecutarse. Si algo sale mal, solo puedes observar.

Reglas:

- Cada tarjeta debe contener UNA sola instrucción.

- Usa solo las operaciones permitidas (ver lista de instrucciones).
 - Numera cada tarjeta en la esquina superior derecha.
 - Una vez entregadas las tarjetas, no puedes modificarlas.
-

UNIDAD DE CONTROL

Eres el director de orquesta. Lees las tarjetas de instrucción una por una, en orden, y le dices al componente correspondiente qué hacer. NO haces cálculos ni almacenas datos — solo coordinas.

Reglas:

- Lee las tarjetas en orden numérico, una a la vez.
 - Indica en voz alta qué componente debe actuar y qué debe hacer.
 - Si una instrucción es ambigua, di "ERROR: instrucción no clara" y detén la ejecución.
 - Si una instrucción pide leer una celda de memoria vacía, di "ERROR: dato no encontrado".
-

ALU (Unidad Aritmético-Lógica)

Eres la calculadora. Solo sabes hacer operaciones aritméticas (+, −, ×, ÷) y comparaciones (>, <, =). Recibes dos números, operas, y devuelves el resultado a quien te lo pida.

Reglas:

- Solo actúas cuando la Unidad de Control te lo indica.
 - Solo puedes operar con los números que te entregan en ese momento.
 - No recuerdas resultados anteriores (no tienes memoria propia).
 - Di el resultado en voz alta.
-

MEMORIA

Eres el almacén temporal. Tienes una hoja con 10 celdas numeradas (0–9). Guardas y entregas datos cuando la Unidad de Control te lo pide.

Reglas:

- Solo actúas cuando la Unidad de Control te lo indica.
 - GUARDAR: escribe el valor en la celda indicada (borra lo que había antes).
 - LEER: di en voz alta el valor almacenado en la celda indicada.
 - Si te piden leer una celda vacía, di "celda vacía".
-

ENTRADA/SALIDA (E/S)

Eres la conexión con el mundo exterior. **Entrada:** le das datos al sistema cuando te los piden (los lees de la "hoja de datos de entrada" que te entrega el docente). **Salida:** escribes el resultado final en la hoja en

blanco (la "pantalla").

Reglas:

- Solo actúas cuando la Unidad de Control te lo indica.
- LEER ENTRADA: di en voz alta el siguiente dato de la hoja de entrada.
- ESCRIBIR SALIDA: escribe el valor indicado en la hoja de pantalla.
- No interpretas los datos, solo los transmites.

Lista de instrucciones permitidas

Proyectar o imprimir para cada grupo:

Instrucción	Significado	Ejemplo
LEER_ENTRADA → celda X	Tomar el siguiente dato de entrada y guardarlo en la celda X de memoria	LEER_ENTRADA → celda 0
GUARDAR valor → celda X	Guardar un valor fijo en la celda X	GUARDAR 3 → celda 5
LEER celda X	Leer y anunciar el contenido de la celda X	LEER celda 0
OPERAR celda X ◦ celda Y → celda Z	Tomar los valores de las celdas X e Y, aplicar la operación ◦ (+, −, ×, ÷), guardar resultado en celda Z	OPERAR celda 0 + celda 1 → celda 2
ESCRIBIR_SALIDA celda X	Escribir el valor de la celda X en la pantalla de salida	ESCRIBIR_SALIDA celda 2

Desarrollo de la actividad

Fase 1 — Formación de grupos y asignación de roles (5 min)

- Formar grupos de 5.
- Cada persona toma una tarjeta de rol al azar (o por preferencia).
- Leer las reglas de su rol en silencio.
- El docente reparte los materiales y la **hoja de datos de entrada** (solo al componente E/S).

Fase 2 — Ronda 1: Problema guiado (20 min)

Problema: *Calcular la densidad poblacional de una especie.*

Datos de entrada (en la hoja de E/S):

- Dato 1: 150 (número de individuos contados)

- Dato 2: 30 (superficie del área muestreada en hectáreas)

Resultado esperado: 5.0 (densidad = $150 \div 30$)

Procedimiento:

1. **(5 min)** El/la Programador/a escribe las instrucciones en las tarjetas. El grupo puede discutir la estrategia, pero **solo el Programador/a escribe**.

Solución esperada (para referencia del docente):

- Tarjeta 1: LEER_ENTRADA → celda 0
 - Tarjeta 2: LEER_ENTRADA → celda 1
 - Tarjeta 3: OPERAR celda 0 ÷ celda 1 → celda 2
 - Tarjeta 4: ESCRIBIR_SALIDA celda 2
2. **(10 min)** Ejecución. El Programador/a entrega las tarjetas a la Unidad de Control y **se queda en silencio**. La Unidad de Control lee cada tarjeta y coordina. Los demás componentes actúan según sus reglas.
 3. **(5 min)** Verificación. ¿La pantalla muestra 5.0? Si hubo errores, identificar dónde falló: ¿instrucción ambigua? ¿celda equivocada? ¿operación incorrecta?

Fase 3 — Ronda 2: Problema autónomo (25 min)

Problema: *Calcular el índice de abundancia relativa de dos especies.*

Datos de entrada:

- Dato 1: 45 (individuos especie A)
- Dato 2: 75 (individuos especie B)

Resultado esperado en pantalla:

- Línea 1: 120 (total de individuos)
- Línea 2: 0.375 (proporción especie A = $45 \div 120$)
- Línea 3: 0.625 (proporción especie B = $75 \div 120$)

Este problema es más complejo: requiere un resultado intermedio (la suma) que se usa en cálculos posteriores. Los estudiantes deben descubrir que necesitan **almacenar el total en una celda** antes de usarlo.

Procedimiento:

1. **(8 min)** Programación. El grupo discute la estrategia, el Programador/a escribe.
2. **(10 min)** Ejecución en silencio del Programador/a.
3. **(7 min)** Verificación y discusión interna del grupo: ¿qué funcionó? ¿qué falló?

Errores comunes esperados (para que el docente observe y use en la discusión):

- Olvidar guardar el total antes de dividir.

- Intentar usar un resultado de la ALU sin guardarlo primero en memoria (la ALU no recuerda).
- Instrucciones ambiguas como "dividir A por el total" sin especificar celdas.
- No numerar las tarjetas → la Unidad de Control no sabe el orden.

Fase 4 — Discusión en sala (10 min)

Reunir a todo el curso. Preguntas para guiar la discusión:

1. **"¿Qué fue lo más difícil?"** → Generalmente: escribir instrucciones suficientemente precisas.
2. **"¿Qué pasó cuando una instrucción era ambigua?"** → El sistema se detuvo o produjo un resultado incorrecto. Un computador real hace lo mismo: si la instrucción no es clara, falla o hace algo inesperado.
3. **"¿La Memoria necesitó 'entender' qué significaban los números?"** → No. Solo los almacenaba y entregaba. Los computadores no "entienden" los datos — eso lo hace el programador al diseñar el algoritmo.
4. **"¿Qué paralelo ven con su futuro trabajo en conservación?"** → Los datos no hablan solos. El análisis depende de las instrucciones (el método, el modelo) que uno aplica. Si el método es impreciso, los resultados son poco confiables — da igual cuántos datos se tengan.
5. **"¿Quién fue el componente más importante?"** → Trampa: ninguno funciona sin los demás. Un computador es un *sistema*.

PARTE 3: CONTROL DIAGNÓSTICO (15 min, sin calificación)

Se puede aplicar al inicio de la cátedra (antes del Bloque A) o al cierre de la sesión (después del laboratorio). La recomendación es aplicarlo al inicio para tener una línea base limpia, no contaminada por la clase.

Control Diagnóstico — Semana 1

Introducción al Análisis de Datos y Programación

Nombre: _____ **Fecha:** _____

Este control NO tiene calificación. Su objetivo es conocer tus conocimientos previos para adaptar el curso a las necesidades del grupo. Responde con honestidad — si no sabes algo, simplemente escribe "no sé".

1. ¿Qué es un computador? Escribe tu propia definición en 2–3 oraciones.

[espacio]

2. Marca con una X las afirmaciones que creas correctas:

- ☐ Un computador solo sirve para navegar por internet y usar redes sociales.
- ☐ Todo computador tiene una unidad que realiza cálculos matemáticos.
- ☐ La memoria RAM almacena información de forma permanente.
- ☐ Un programa es un conjunto de instrucciones que el computador ejecuta en orden.
- ☐ Los computadores pueden "pensar" de la misma forma que los seres humanos.
- ☐ El sistema operativo es un programa que administra el hardware.

3. ¿Qué es un algoritmo? Si no conoces la palabra, intenta adivinar a partir de su sonido o contexto.

[espacio]

4. ¿Has programado alguna vez? (Marca una opción)

- ☐ Nunca.
- ☐ He usado Scratch, Logo u otro lenguaje visual/bloques.
- ☐ He escrito algo de código en un lenguaje de texto (¿cuál? _____).
- ☐ Programo regularmente.

5. ¿Qué significa el número 1010 en sistema binario? (Si no sabes, escribe "no sé".)

[espacio]

6. ¿Has usado alguna herramienta de inteligencia artificial (por ejemplo, ChatGPT, Claude, Copilot, generadores de imágenes)? Si la respuesta es sí, describe brevemente para qué la usaste.

[espacio]

7. Imagina que recibes un conjunto de datos con los conteos de 20 especies de aves en 10 sitios de muestreo. ¿Qué preguntas te gustaría responder con esos datos? Escribe al menos dos.

[espacio]

8. ¿Qué esperas aprender en este curso? (respuesta libre)

[espacio]

Clave de lectura del diagnóstico (para el docente)

Pregunta	Qué evalúa	Señales de alerta / oportunidad
1	Concepto intuitivo de computador	Si nadie menciona "instrucciones" o "cálculos", la clase del Bloque D fue necesaria

Pregunta	Qué evalúa	Señales de alerta / oportunidad
2	Conocimientos factuales básicos	Correctas: 2, 4, 6. Si muchos marcan RAM permanente (3), reforzar en Semana 2
3	Noción de algoritmo	La mayoría no conocerá el término; útil como línea base para Semana 4
4	Experiencia previa en programación	Calibrar la velocidad de la Sección 2 según la distribución
5	Conocimiento de binario	Respuesta: 10 (decimal). Línea base para Semana 2
6	Familiaridad con IA	Crucial para calibrar la Semana 6 y el nivel de la discusión sobre LLMs
7	Pensamiento analítico / ecológico	Identifica a estudiantes que ya piensan en preguntas de investigación
8	Motivación y expectativas	Información cualitativa para ajustar el tono del curso

Notas pedagógicas

Errores frecuentes y cómo aprovecharlos

El laboratorio analógico está diseñado para que los estudiantes *fallen*. Los errores más comunes son oportunidades de aprendizaje:

- **Instrucciones en lenguaje natural** ("calcula la densidad"): el "computador" no sabe qué es densidad. → Lección: los computadores no tienen contexto ni sentido común.
- **Omitir pasos intermedios** (dividir sin guardar primero el total): → Lección: la ALU no tiene memoria; los resultados se pierden si no se almacenan explícitamente.
- **Instrucciones fuera de orden o sin numerar**: → Lección: la secuencia importa. Un programa es una lista *ordenada* de instrucciones.
- **El Programador/a intenta intervenir durante la ejecución**: → Lección: una vez compilado/lanzado, el programa corre solo. Si tiene errores, hay que detenerlo, corregir y volver a ejecutar (*debugging*).

Extensión opcional (si hay tiempo o grupos avanzados)

Ronda 3: Introducir un condicional.

Agregar una instrucción nueva a la lista:

Instrucción	Significado
SI celda X > valor ENTONCES ir a tarjeta N, SINO ir a tarjeta M	Bifurcación condicional

Problema: Si la densidad poblacional es menor a 1.0 individuo/ha, escribir en la pantalla "ESPECIE EN RIESGO". Si no, escribir "POBLACIÓN ESTABLE".

Esto anticipa los condicionales de la Semana 10 y muestra por qué los computadores pueden "tomar decisiones" sin realmente decidir nada.

Conexión con el resto del curso

Concepto de esta semana	Dónde se profundiza
Instrucciones precisas y secuenciales	Semana 4 (Algoritmos), Semana 8 (Python)
Componentes del hardware (ALU, memoria, E/S)	Semana 2 (Representación binaria)
La ALU no tiene memoria propia	Semana 8 (Variables), Semana 12 (Scope)
El computador no "entiende"	Semana 6 (LLMs: ¿entienden o predicen?)
Errores por ambigüedad	Semana 4 (Pseudocódigo), Semana 8 (Sintaxis)